

Kafka

Apache Kafka là một nền tảng streaming sự kiện phân tán, có khả năng mở rộng để xử lý các pipeline dữ liệu thời gian thực với quy mô rất lớn. Kafka được tạo ra vào năm 2011 tại LinkedIn, được viết bằng Java và Scala, và được đặt tên là Kafka vì đây là một hệ thống được tối ưu cho việc ghi dữ liệu. Ta có thể hình dung việc sử dụng Kafka để xây dựng một dashboard giống như Google Analytics: khi một sự kiện xảy ra, chẳng hạn như một lượt truy cập website, Producer API sẽ tạo ra một bản ghi mới. Các bản ghi này được lưu xuống đĩa dưới dạng một log có thứ tự và bất biến, gọi là topic, có thể tồn tại vĩnh viễn hoặc bị xóa khi không còn cần thiết. Các topic được phân tán và sao chép trong một cluster gồm nhiều máy chủ gọi là broker, điều này giúp Kafka có khả năng chịu lỗi và mở rộng để đáp ứng mọi khối lượng công việc. Ở phía ngược lại, nhiều consumer có thể đăng ký để nhận dữ liệu này; họ có thể đọc thông điệp mới nhất giống như trong một hàng đợi, hoặc đọc toàn bộ log của topic và lắng nghe các cập nhật theo thời gian thực. Ngoài ra, Kafka còn cung cấp Streams API rất mạnh mẽ, cho phép biến đổi và tổng hợp các topic trước khi dữ liệu đến tay consumer. Mặc dù có vẻ tương tự các message broker như RabbitMQ, Kafka có thể xử lý throughput lớn hơn nhiều và đặc biệt phù hợp cho các ứng dụng streaming dữ liệu. Trên thực tế, Kafka đang được các công ty như Lyft sử dụng để thu thập và xử lý dữ liệu định vị, Spotify và Netflix dùng cho xử lý log, còn Cloudflare dùng cho phân tích thời gian thực. Để bắt đầu, bạn tải Kafka về và sử dụng các công cụ như ZooKeeper hoặc KRaft để quản lý cluster. Sau khi môi trường được thiết lập, bạn có thể tạo topic đầu tiên; mỗi topic là một log sự kiện được sắp xếp theo thứ tự, trong đó mỗi sự kiện có key, value, timestamp và có thể kèm theo metadata hay header. Khi publish các sự kiện vào topic, Kafka đảm bảo rằng mọi consumer của cùng một topic sẽ luôn đọc các sự kiện theo đúng thứ tự ban đầu. Consumer có thể đọc sự kiện mới nhất, đọc toàn bộ log từ đầu hoặc chỉ một phần dữ liệu thông qua offset. Ở mức nâng cao hơn, Kafka Streams API cho phép thực hiện các biến đổi không trạng thái như lọc sự kiện, hoặc biến đổi có trạng thái như tổng hợp nhiều sự kiện trong một khoảng thời gian nhất định, từ đó giúp bạn quản lý các luồng dữ liệu thời gian thực với quy mô gần như không giới hạn.

NGINX

Nginx có thể được xem như một công trung gian đứng giữa Internet và hạ tầng back-end của bạn. Khi bạn truy cập một ứng dụng web, nơi đầu tiên mà yêu cầu của bạn đi tới chính là web server; nhiệm vụ của nó là kiểm tra tài nguyên được yêu cầu, xác định vị trí của tài nguyên đó trên máy chủ và sau đó trả về cho người dùng dưới dạng phản hồi. Thực tế, nếu bạn mở tab Network trong Chrome DevTools và xem header của bất kỳ response nào, rất có thể bạn sẽ thấy server được ghi là Nginx. Nginx cực kỳ phổ biến với các website có lưu lượng truy cập lớn vì kiến trúc hướng sự kiện của nó cho phép xử lý hơn 10.000 kết nối đồng thời. Ngoài ra, Nginx thường được sử dụng như một reverse proxy, đóng vai trò như “đền giao thông” để phân phối lưu lượng đến nhiều máy chủ back-end khác nhau, đồng thời cung cấp các cơ chế bảo mật và caching nhằm cải thiện hiệu năng. Trong hầu hết các trường hợp, Nginx được cài đặt trên máy chủ Linux và file

cấu hình của nó nằm trong thư mục /etc. Bạn có thể tùy chỉnh hành vi của máy chủ bằng cách định nghĩa các directive; một directive đơn giản là một cặp key–value, còn nếu đi kèm với dấu ngoặc nhọn thì được gọi là context, bên trong chứa nhiều directive khác. Ở global context, bạn có thể thiết lập các thông tin như user chạy Nginx hoặc vị trí lưu error log, nhưng phần lớn cấu hình thường nằm trong http context. Một trong những vai trò quan trọng nhất của Nginx là phục vụ nội dung tĩnh như hình ảnh và file HTML; trong http context, bạn sẽ định nghĩa một hoặc nhiều server, mỗi server được phân biệt bởi cổng (port) mà nó lắng nghe. Nginx theo dõi mọi request gửi đến server và có thể ghi lại trong access log, nhưng điều quan trọng nhất là chỉ ra cho server biết vị trí các file gốc thông qua location context. Khi người dùng truy cập vào domain, Nginx sẽ biết cách tìm file HTML trên hệ thống file, và bạn cũng có thể cấu hình thêm các location khác để ánh xạ các yêu cầu hình ảnh tới thư mục images. Những cấu hình phổ biến khác trong server config bao gồm chứng chỉ SSL, rewrite URL và định tuyến tới proxy server. Khi bạn thay root bằng proxy_pass, bạn có thể chuyển tiếp request tới một máy chủ hoàn toàn khác trên Internet; lúc này, bạn đã tạo ra một reverse proxy có khả năng caching, ẩn danh và cân bằng tải.

GraphQL

GraphQL là một ngôn ngữ truy vấn dùng để đọc và thay đổi (mutate) dữ liệu trong các API. Với vai trò là một lập trình viên back-end, GraphQL cung cấp cho bạn một hệ thống kiểu (type system) cho phép mô tả schema của dữ liệu, từ đó trao cho phía front-end khả năng khám phá và yêu cầu chính xác những dữ liệu mà họ cần. Truyền thống, các lập trình viên web thường tiêu thụ API theo mô hình REST, nơi mỗi thực thể dữ liệu được gắn với một URL riêng trên server và khi nhận request, API sẽ trả về toàn bộ payload dữ liệu của thực thể đó. Cách này nghe có vẻ đơn giản, nhưng có hai nhược điểm chính: trong nhiều trường hợp, ta cần dữ liệu từ nhiều thực thể cùng lúc nên mỗi request lại bị thiếu dữ liệu cần thiết (under-fetching); ngược lại, đôi khi ta chỉ cần một phần nhỏ của thực thể nhưng vẫn phải nhận toàn bộ dữ liệu (over-fetching), gây lãng phí tài nguyên. Khác với REST sử dụng nhiều URL, một API GraphQL chỉ có một điểm vào (entry point) duy nhất, và dữ liệu được truy vấn bằng cách mô tả cấu trúc mong muốn thông qua cú pháp phản ánh trực tiếp hình dạng JSON trả về. Lập trình viên front-end sẽ mô tả dữ liệu họ cần, còn lập trình viên back-end sẽ viết logic để xử lý và trả về kết quả, tất cả đều diễn ra với cú pháp có thể làm việc với bất kỳ ngôn ngữ lập trình nào. Bạn có thể bắt đầu bằng việc định nghĩa schema với các object tùy chỉnh thông qua từ khóa type; mỗi type có thể chứa nhiều field như id duy nhất (có thể đánh dấu là bắt buộc), các giá trị kiểu số nguyên hoặc chuỗi, và các mối quan hệ giữa các type, chẳng hạn một creator có thể có nhiều video, còn một video thì thuộc về một creator. Mỗi API GraphQL đều có một type Query, đóng vai trò là điểm vào chính để người dùng đọc dữ liệu, cho phép truy vấn danh sách video hoặc một người dùng cụ thể theo id. Ngoài ra, GraphQL còn hỗ trợ type Mutation để định nghĩa cách dữ liệu được thay đổi trong API. Sau khi triển khai, bất kỳ lập trình viên nào sử dụng API này đều có thể khám phá nó với sự hiểu biết đầy đủ về các truy vấn và thực thể dữ liệu có sẵn, đồng thời được hỗ trợ các công cụ

tự động gợi ý (autocomplete) khi viết truy vấn trong trình soạn thảo, giúp việc phát triển trở nên nhanh chóng và hiệu quả hơn.

Kubernetes

Kubernetes là một công cụ dùng để quản lý và tự động hóa các khối lượng công việc được container hóa trong môi trường điện toán đám mây. Có thể hình dung Kubernetes giống như một nhạc trưởng, còn mỗi nhạc công trong một dàn nhạc chính là các container Docker riêng lẻ; để tạo ra một bản nhạc hài hòa, cần có nhạc trưởng điều phối các nhạc công và giữ nhịp độ, và trong trường hợp này nhạc trưởng chính là Kubernetes. Với những ứng dụng có lưu lượng thay đổi lớn, chẳng hạn như một ứng dụng giao dịch chứng khoán, khi thị trường đóng cửa thì tải hệ thống rất thấp, nhưng khi thị trường mở cửa thì ứng dụng phải xử lý hàng triệu giao dịch cùng lúc, Kubernetes sẽ đóng vai trò điều phối hạ tầng để đáp ứng sự thay đổi đó. Nó có khả năng mở rộng các container trên nhiều máy khác nhau và nếu một container hay máy gặp sự cố, Kubernetes sẽ tự động thay thế bằng một phiên bản mới. Một hệ thống được triển khai trên Kubernetes được gọi là một cụm (cluster), trong đó bộ não điều khiển là control plane, cung cấp API server để xử lý các yêu cầu quản lý cụm từ bên trong lẫn bên ngoài, đồng thời sử dụng một cơ sở dữ liệu key-value riêng gọi là etcd để lưu trữ thông tin quan trọng về trạng thái của cluster. Các tài nguyên mà control plane quản lý là các máy làm việc gọi là node, mỗi node chạy một tiến trình nhỏ tên là kubelet để giao tiếp với control plane trung tâm. Bên trong mỗi node có nhiều pod, đây là đơn vị triển khai nhỏ nhất trong Kubernetes, có thể hiểu là một nhóm container chạy cùng nhau. Khi khối lượng công việc tăng lên, Kubernetes có thể tự động mở rộng theo chiều ngang bằng cách thêm node vào cluster, đồng thời xử lý các vấn đề phức tạp như mạng, quản lý bí mật, lưu trữ bền vững và nhiều yếu tố khác. Hệ thống được thiết kế với tính sẵn sàng cao, một trong những cách đạt được điều này là duy trì các replica set, tức là một tập hợp các pod hoặc container luôn sẵn sàng hoạt động. Với vai trò là lập trình viên, bạn sẽ định nghĩa các đối tượng bằng tệp YAML để mô tả trạng thái mong muốn của cluster, ví dụ như một deployment Nginx với ba pod trong replica set, kèm theo các cấu hình chi tiết về container, volume, cổng mạng và hành vi hoạt động. Từ những cấu hình này, Kubernetes sẽ tự động triển khai, mở rộng và đảm bảo các container luôn hoạt động ổn định và khỏe mạnh.

MongoDB

MongoDB là một trong những cơ sở dữ liệu hướng tài liệu phổ biến và tiên tiến nhất trên thế giới. Nó được tạo ra vào năm 2007, sau khi đội ngũ tại DoubleClick — một công ty phục vụ khoảng 400.000 quảng cáo mỗi giây — gặp phải các vấn đề về khả năng mở rộng và tính linh hoạt khi sử dụng các hệ quản trị cơ sở dữ liệu truyền thống. Điều này đã thúc đẩy họ thiết kế một hệ thống mới, trong đó toàn bộ dữ liệu được lưu trữ dưới dạng các tài liệu giống JSON và được tổ chức thành các collection có thể truy vấn. Không giống như bảng trong cơ sở dữ liệu quan hệ, MongoDB không bắt buộc phải có schema cố định cho một collection, cho phép bạn thay đổi và phát triển cấu trúc dữ

liệu rất nhanh mà không cần thực hiện các bước migration phức tạp. Quan trọng hơn, MongoDB cho phép lưu trữ những dữ liệu thường xuyên được ứng dụng truy cập cùng nhau vào một tài liệu duy nhất, giúp các thao tác đọc trở nên cực kỳ nhanh vì không cần thực hiện các phép join, giống như việc có sẵn một chiếc xe đã được lắp ráp hoàn chỉnh thay vì phải ghép từng bộ phận rời rạc. Cách tiếp cận này cũng giúp cơ sở dữ liệu dễ dàng mở rộng theo chiều ngang thông qua sharding, bởi mỗi collection là một đơn vị tự chứa và rất phù hợp cho các hệ thống phân tán, đúng với triết lý “document-oriented” được thiết kế để xử lý khối lượng công việc cực lớn. Khi tạo một document mới trong MongoDB, hệ thống sẽ tự động gán cho nó một ObjectId duy nhất trong collection, bên trong document bạn có thể định nghĩa nhiều field với các kiểu dữ liệu khác nhau như chuỗi, mảng, object và nhiều loại khác. Để làm việc với dữ liệu trong bất kỳ ngôn ngữ lập trình nào, MongoDB cung cấp Query API cho các thao tác đọc, ghi cơ bản cũng như các truy vấn và biến đổi dữ liệu phức tạp; ngoài ra bạn có thể tạo các secondary index để tăng tốc đáng kể những truy vấn phổ biến, đồng thời hỗ trợ cả truy vấn không gian địa lý nhằm tìm các document gần một vị trí cụ thể. Bên cạnh truy vấn, MongoDB còn hỗ trợ aggregation pipeline để nhóm và xử lý nhiều document, sau đó tổng hợp chúng thành một kết quả duy nhất. MongoDB là mã nguồn mở và có thể tự triển khai, tuy nhiên cách nhanh nhất để bắt đầu là sử dụng gói miễn phí trên MongoDB Atlas, nơi hệ thống tự động mở rộng, cung cấp giao diện quản lý trực quan và nhiều tính năng mạnh mẽ như full-text search dựa trên Apache Lucene. Ngoài ra, MongoDB còn ra mắt tùy chọn serverless giúp bạn chỉ trả chi phí đúng theo tài nguyên sử dụng, cùng với các tính năng quan trọng khác trên Atlas như trigger để dễ dàng chạy các hàm serverless khi dữ liệu thay đổi, và Realm — một nền tảng tương tự Firebase giúp đồng bộ dữ liệu thời gian thực với ứng dụng front-end.

Vim

Vim là một trình soạn thảo văn bản để viết mã nguồn, trong đó bạn điều hướng và thao tác hoàn toàn bằng bàn phím thay vì dùng chuột. Nó được xây dựng dựa trên trình soạn thảo Unix nguyên bản là vi, ra đời từ năm 1976, sau đó được cải tiến thành vim (vi improved) vào năm 1991 với rất nhiều nâng cấp. Trong thời đại của các IDE hiện đại, bạn có thể thắc mắc vì sao lại cần dùng một trình soạn thảo dựa trên bàn phím, nhưng khi bạn viết code mỗi ngày thì lý tưởng nhất là tay luôn đặt trên bàn phím; mỗi lần đưa tay sang chuột đều làm giảm năng suất. Học lập trình với Vim giống như học chơi một nhạc cụ: ban đầu sẽ khá đau đầu và khó chịu, nhưng sự khó khăn đó sẽ giúp bạn chỉnh sửa code chính xác và hiệu quả hơn rất nhiều về lâu dài. Vim chạy trực tiếp trong terminal và được cài sẵn trên hầu hết các máy, vì vậy sớm hay muộn bạn cũng sẽ vô tình “rơi” vào Vim mà không biết cách thoát; khi đó bạn có thể gõ :q để thoát nếu chưa chỉnh sửa, :q! để thoát và bỏ mọi thay đổi, hoặc :wq để lưu rồi thoát. Khi mở một file bằng Vim, bạn mặc định ở normal mode, và có thể chuyển đổi giữa các chế độ như insert, visual và command line tùy mục đích sử dụng; bạn di chuyển con trỏ bằng các phím mũi tên hoặc các phím h, j, k, l, xóa ký tự bằng x, xóa cả dòng bằng dd, hoàn tác bằng u, hiển thị số dòng bằng :set number, nhảy đến dòng bất kỳ bằng :số_dòng, chỉnh sửa nội dung

bằng cách vào insert mode với phím i, rồi quay lại normal mode bằng phím Esc. Vim cũng hỗ trợ sao chép—dán từ clipboard hệ thống, lưu file, và thậm chí chạy chương trình trực tiếp từ trong editor. Không cần phải bỏ VS Code để tận dụng sức mạnh của Vim, bạn có thể cài extension VS Code Vim để sử dụng các phím tắt và triết lý của Vim ngay trong editor quen thuộc. Vim được xem là một kỹ năng “evergreen”, học một lần có thể dùng suốt sự nghiệp, giúp bạn làm việc nhanh hơn, hiệu quả hơn và tập trung nhiều hơn vào việc viết code thay vì thao tác chỉnh sửa, với các lệnh điều hướng như h j k l, chỉnh sửa trong dòng bằng i, I, a, A, và chỉnh sửa trực tiếp trong command mode bằng x và r, từ đó cho phép bạn kiểm soát trình soạn thảo một cách linh hoạt, chính xác và hoàn toàn bằng bàn phím.

Session vs Token Authentication

Xác thực người dùng có hai cách chính để thực hiện, đó là session và token. Cách tiếp cận truyền thống trên web là sử dụng session phía server dựa trên cookie. Quá trình bắt đầu khi người dùng nhập tên đăng nhập và mật khẩu rồi gửi lên server; server sẽ xác thực thông tin này, tạo một session trong cơ sở dữ liệu và trả về một session ID. Session ID này được lưu trong cookie của trình duyệt, tức là nơi lưu các cặp key—value và sẽ được gửi kèm theo mỗi request tiếp theo đến server. Dựa vào đó, server có thể trả về nội dung phù hợp với người dùng đang đăng nhập, nghĩa là giữa client và server tồn tại một session có trạng thái (stateful). Cách này hoạt động tốt nhưng có một số nhược điểm, chẳng hạn như nguy cơ bị tấn công CSRF (Cross-Site Request Forgery), khi kẻ tấn công dụ người dùng đã đăng nhập truy cập vào một trang khác để thực hiện những hành động họ không chủ ý như thanh toán hay đổi mật khẩu. Dù rủi ro này khá thấp nếu dùng framework hiện đại, vấn đề lớn hơn là server phải lưu session ID trong bộ nhớ hoặc cơ sở dữ liệu, và trong các ứng dụng cloud ngày nay thường được scale theo chiều ngang, điều này có thể trở thành nút thắt hiệu năng nghiêm trọng.

Để giải quyết vấn đề đó, xác thực dựa trên token ra đời, dù nó cũng mang theo những thách thức riêng. Quy trình ban đầu tương tự: client gửi thông tin đăng nhập lên server, nhưng thay vì tạo session ID, server sẽ sinh ra một JSON Web Token (JWT) được ký bằng khóa bí mật. Token này được gửi về trình duyệt và thường được lưu trong local storage. Ở các request tiếp theo, token sẽ được đính kèm trong header Authorization với tiền tố Bearer. Khi đó, server chỉ cần kiểm tra chữ ký của token mà không cần truy vấn cơ sở dữ liệu, điều này hiệu quả hơn rất nhiều trong các hệ thống phân tán trên cloud. Tuy nhiên, token vẫn có thể bị đánh cắp, khó bị vô hiệu hóa, và không phù hợp để xác thực người dùng trong các tác vụ nền phía server. Điều quan trọng nhất cần hiểu là với session, trạng thái xác thực được quản lý ở phía server, còn với token thì trạng thái đó được quản lý ở phía client.

